Spring Boot 3 & Microservices

1. Dependency Injection: A design pattern where objects get their dependencies from the Spring container instead of creating them. It improves loose coupling, testability, and maintainability.

2. Auto-Configuration: Spring Boot automatically configures beans based on classpath dependencies and application properties, reducing boilerplate configuration.

3. @Controller vs @RestController: @Controller is used for MVC and returns views, while @RestController combines @Controller and @ResponseBody to return JSON/XML responses.

4. Distributed Transactions: In microservices, 2PC is avoided. Saga pattern is used where each service has local transactions and compensating actions.

Microservices

1. Circuit Breaker: A resilience pattern that stops calling a failing service after a threshold and provides fallback, preventing cascading failures.

2. Singleton vs Prototype Scope: Singleton creates one bean per Spring container; Prototype creates a new instance every time it is requested.

3. Spring Boot Native Images: Built using GraalVM, they provide faster startup, lower memory usage, and are ideal for cloud-native and serverless apps.

4. Configuration Management: Use centralized config via Spring Cloud Config, environment variables, or Kubernetes ConfigMaps and Secrets.

System Design & SQL

1. N+1 Problem: Occurs when one query fetches parent data and multiple queries fetch children. Solved using fetch joins, EntityGraph, or batch fetching.

2. CAP Theorem: States that a distributed system can guarantee only two of Consistency, Availability, and Partition Tolerance. Traditional RDBMS prefer CA.

3. Indexing vs Partitioning: Indexing improves query performance; partitioning splits large tables for scalability and maintenance.

4. Rate Limiter: Controls API traffic using algorithms like token bucket or sliding window, often implemented with Redis.

5. SQL vs NoSQL: MongoDB is preferred for flexible schema and scalability; PostgreSQL is better for ACID transactions and complex queries.

Modern Java (17–21)

1. Virtual Threads: Lightweight threads managed by JVM that improve scalability for I/O-bound workloads compared to platform threads.

2. Records: Immutable data classes that automatically generate constructor, getters, equals, hashCode, and toString.

3. Sealed Classes: Restrict which classes can extend a class or interface, improving domain modeling and security.

4. Pattern Matching for switch: Enhances switch by allowing type checks, reducing casting and boilerplate.

5. var vs Dynamic Typing: var performs compile-time type inference; Java remains statically typed.

Core Java & Internals

1. HashMap Internals: Uses array of buckets with linked list/tree structure. From Java 8, buckets convert to red-black tree when collisions increase.

2. Fail-fast vs Fail-safe Iterators: Fail-fast throws ConcurrentModificationException; fail-safe works on a cloned collection.

3. Try-with-resources: Automatically closes resources that implement AutoCloseable, preventing resource leaks.

4. CompletableFuture vs Future: CompletableFuture supports non-blocking async operations and chaining.

5. G1GC vs Parallel GC: G1 GC focuses on predictable low pause times; Parallel GC focuses on throughput.

6. String Immutability: Ensures thread safety, security, and enables string pooling.

7. volatile vs synchronized: volatile ensures visibility across threads; synchronized provides mutual exclusion and atomicity.